Distributed and local computing on low-end devices from data privacy and decentralization perspectives

1st Noyan Tendikov
School of Cybersecurity
Astana IT University
Astana, Kazakhstan
ORCID: 0009-0009-7251-8830

2nd Shynbolat Unaibayev
School of Cybersecurity
Astana IT University
Astana, Kazakhstan
ORCID: 0009-0005-3192-8906

3rd Sagyndyk Suleimanov
School of Cybersecurity
Astana IT University
Astana, Kazakhstan
ORCID: 0009-0004-3537-4368

4th Angsar Kabdikarim
School of Cybersecurity
Astana IT University
Astana, Kazakhstan
ORCID: 0009-0002-7851-2076

5th Daulet Zhumat
School of Cybersecurity
Astana IT University
Astana, Kazakhstan
ORCID: 0009-0002-3488-0360

6th Bakdaulet Zhaksylyk
School of Cybersecurity
Astana IT University
Astana, Kazakhstan
ORCID: 0009-0003-7903-0180

***Abstract*—The current geopolitical times demonstrate that a limited number of dominant actors hold the power to unilaterally block access to critical technologies or enforce systemic censorship. On the other hand, security regulations within governmental and military groups strictly prohibit the use of external services to prevent data leakage and ensure operational independence. In these conditions, we developed a concept of a distributed network that is focused on decentralized computation and digital sovereignty on low-end devices. In order to analyze its applicability and identify possible limitations, we collected a partial dataset of publicly known Android-compatible smartphone models of 1745 unique samples with their characteristics. The analysis results demonstrate that about 66.78 percent, which amounts to 989 out of 1481 devices, received their latest security patches 3 or more years ago. Moreover, some of them cannot be updated to the latest major versions, get backport updates, fix compatibility issues with certain applications, and patch the kernel due to limitations of hardware and software sides. Therefore, such legacy devices are potentially dangerous and problematic to use as nodes in a public network, but they can only be applied with associated risks in closed, local, and air-gapped environments.**

Keywords—distributed systems, small models, mobile inference, low-end devices, decentralized network, data privacy, defense-in-depth, trusted and confidential computing

I. Introduction

The current state of hardware prices and associated shortages is directly linked to the race around artificial intelligence (AI). Cloud services and AI-oriented organizations demand exponential increases in computational resources for their IT infrastructures. In this regard, suppliers and manufacturers act as cartels, regulating prices and monopolizing their efforts only around enterprise customers. These market conditions lead to technofeudalism, where major corporations dictate the rules of "the game", forcing users into their ecosystems through vendor lock-in and subscription-based models to eliminate individual ownership. Moreover, centralized data centers represent single points of failure that are vulnerable to significant impact from physical damage to specific locations or service blockages from state policies. Furthermore, many hardware technologies and their production processes remain strictly proprietary, resulting in a lack of competition between actors and reducing overall production rates.

Therefore, our research explores the possibilities of participation for small groups and single actors in that race from different perspectives, such as optimized training and inference, data collection and processing, combining low computing devices with cost-effectiveness of power consumption, and ensuring data privacy without exposure to third parties. Firstly, the literature review covers three major topics: application of decentralized networks for general-purpose tasks (e.g., data storage, in-memory processing, and runtime execution); introduction of optimization techniques designed to democratize AI; analysis of general security aspects. Secondly, the methodology shows the concept of the proposed distributed network by covering its tokenomics, resistance, and fairness to prevent centralization of nodes in pools. Thirdly, the results describe our dataset of Android-compatible models of smartphones with their characteristics. We cover security-related issues for self-hosting users and local company infrastructures, including the absence of packet validation, old security updates, session and token-based authentication, mechanisms to mitigate data spamming, and so on. The discussion and conclusion sections identify possible limitations of this approach regarding network constraints, physical limitations of available devices, and data transmission latency.

II. Literature review

A. Distributed computing and blockchain-based systems

The development of the internet and the advancement of networking technologies provided capabilities for computing systems to operate across geographically distributed locations, accelerating information exchange, the accumulation of collective knowledge, and hosting both public or private digital services. For example, crypto networks as overlay networks allow to conduct direct financial transactions and peer-to-peer (P2P) connections between multiple nodes. Several of these decentralized networks leverage blockchain as an immutable data structure to ensure trust and data integrity [1]. Besides, there are alternative architectures and structures with other characteristics (e.g., graphs or trees [2]) and without such limitations of blockchain:

- Versioning or migration during hard forks needs branching of the chain.
- Separation of inner-chain and inter-chain to support scaling of nodes [3].
- Block propagation / relaying time requires prevention of redudant data transmissions and shorter distances to be constant [4].

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Distributed systems are characterized by having consensus protocols and zero-knowledge proofs which are both mechanisms for validating or verifying computational correctness and compliance with predefined set of rules [5]. Their development and approbation are complex processeses involving rigorous vulnerability analysis and formal verifications [6]. For example, the framework proposed in [7] integrates proof assistants with language models to automate the formal and mathematical verification of protocols. However, experts raise ecological concerns by shifting towards cheap renewable energy and energy-efficient optimizations, because certain consensus mechanisms such as Proof of Work (PoW) require intensive redundant computations that consume more power for protocol security than providing tangible utility for humanity [8], [9]. Therefore, [10] and [11] advocate for Proof of Useful Work (PoUW), which provides practical applications for model training, algorithmic optimization, data labeling, and other valuable tasks. Various factors within certain crypto networks (e.g., dynamic difficulty, halving events, and market hype) have shaped their economics to token holding and the centralization of power within large pools. For instance, FruitChains tries to mitigate it by guaranteeing fair reward distribution proportional to computational power [12]. Moreover, Auditable Proof-of-Work (APoW) allows nodes to cryptographically prove their efforts, audit participants, and detect distributed attacks [13].

Table 1 describes how mechanism of distributed ledgers can be used to decentralize traditional centralized IT infrastructures, thereby preventing the risk of outages from GitHub, Cloudflare, and other global services.

TABLE I. DECENTRALIZED SERVICES OVER CENTRALIZED ONES

Authors	***Concepts***
Stoz and Debatty [14]	Decentralized architecture leveraging IPFS (InterPlanetary File System) for software package management.
Ince et al. [15]	System for Linux ecosystem in which package providers act as nodes for reproducible building of software packages.
Costa et al. [16] and Scheid et al. [17]	Distributed repositories for open-source projects and software packages with peer-to-peer connections.
Hammad et al. [18] and Nizamuddin et al. [19]	Decentralized and immutable version control systems to secure the software development lifecycle.
Giamouridis et al. [20]	Blockchain-based DNS (BDNS) against chokepoints of most DNS servers/services.

B. *Optimizing computing and small models on edge devices*

While historical trends indicate that computational demands have continuously increased since the inception of deep learning and throughout the current large-scale era of AI [21], current hardware is reaching a bottleneck due to physical limits on transistor size and requires extensive methods from Table 2.

TABLE II. OVERCOMING HARDWARE SIDE LIMITATIONS

Authors	***Concepts***
Absar et al. [22] and Jelinčič et al. [23]	Accelerating AI computing via Neural Processing Units (NPUs), Tensor Processing Units (TPUs), and hardware that is designed for specific architectures of models.
Taalas [24]	They propose a solution for mapping generative models directly onto ASIC (Application-Specific Integrated Circuit) hardware for inference. This approach increases tokens per second and reduces manufacturing costs by eliminating the need for expensive general-purpose hardware (e.g., GPU). However, this method prevents on-the-fly model updates because it requires producing a new separate chip.
Zhao et al. [25]	Future perspectives of integrating quantum computing for optimized matrix and other operations in AI. Also, it gives understanding that engineers have to explore alternative forms of computations (e.g., non-Von Neumann architectures, analog computing instead of digital one due to lower power consumption, etc.).
McMahon [26]	Optical hardware offers advantages over electronic one via massive bandwidth and speed of light capabilities similarly to fiber-optic technologies. These systems can reduce thermal/heat output due to low resistance and the charge-neutrality of photons. Furthermore, co-packaged optics can act as a transitional layer between photons and electrons to bypass the overhead of packet-based networking.
Li et al. [27], Amatuni et al. [28], Morán et al. [29], and Lee et al. [30]	Firstly, offloading partial neural network inference or other efforts from data centers to edge devices can reduce general environmental impact (energy consumption, carbon emissions, and water usage). Secondly, they advocate for extending device lifecycles through second-hand markets. Thirdly, hardware efficiency can be improved by optimizing power states through checkpoints, transferring produced heat for other utilities, and undervolting to lower passive idle costs.
Oellermann et al. [31]	Developments in open hardware ecosystems for democratizing high-performance computing, such as RISC-V instruction set, reprogrammable FPGAs (Field-programmable Gate Array), Vulkan backend instead of proprietary CUDA, and customizable microcontrollers.

DeepSeek demonstrated that training a 671B parameter SOTA (state-of-the-art) model can be completed even with 2.788 million hours of H800 GPU (approximately 5.6 million USD at 2024 market rates) and knowledge distillation techniques [32]. Larger organizations are expanding into broader fields of digital information (video, audio, and images) for multimodal capabilities, involving greater scale and density of data which requires a corresponding increase in resources. Maintaining IT infrastructure for such tasks (e.g., training models, processing large datasets, and running model inference) requires accounting for the state of hardware (e.g., service lifetime, wear, and degradation) due to multiple factors like overheating or throttling under high-intensity loads and environmental hazards. In this case, silicon chips have weaknesses under cosmic rays and radiation that lead to unpredictable or non-deterministic behavior, while diamonds and their synthetic variations are used in radiation-hardened electronics [33].

On the other hand, the logical and software side includes different optimization strategies, of which Table 3 presents only a small portion of the overall landscape.

TABLE III. OPTIMIZING AND PROTECTING SOFTWARE SIDE

Concepts	***Examples***
Green computing and power efficient programming languages	The growth of the AI sector contributes to higher carbon emissions from burning gas, coal, and oil due to increased electricity consumption [34]. Reducing the power consumption and environmental footprint of computing involves optimizing algorithms (following Big-O notation), adopting energy-efficient programming languages (complied languages instead of interpreted ones), increasing hardware utilization (concurrency and parallelism), and restarting programs on-demand instead of background running [35], [36]. However, Kempen et al. demonstrate that the memory usage of a programming language (PL) affects power consumption (e.g., semantics of immutable objects require creating a new object in memory instead of mutating the previous one) [37]. Also, specific implementations of PL with the same syntax can differ in their runtime (e.g., PyPy has JIT optimizations while CPython does not) [37, pp. 4–5].
Optimizing input/output context	The input/output context window of language models is limited and requires increasing amounts of computation. Therefore, experts suggest possible techniques to address these issues: - RAG (retrieval-augmented generation) and knowledge graph to search relevant information by indexed chunks or components of graph (vertices and edges). - Using Mandarin Chinese instead of English [38], context minimizers with constraint expressiveness [39], and combining multiple languages with low entropy in reasoning blocks can reduce token consumption during model inference. - Prompting in structured data formats like YAML and JSON without whitespaces instead of plaintext [40, p. 9]. - Allowing language model mixing multiple PL depending on specifics of tasks. - Concise specialized PL (e.g., APL, SQL, etc.) and short identifiers like in C standard library instead of being too expressive and human-readable. - Allowing language model to directly call CLI (command line interface) and shell commands instead of forming JSON requests to MCP (model context protocol) server. Also, Paramanayakam et al. demonstrate that selectively scoping available API tools for function calling improves agentic task-completion accuracy and reducing execution time [41]. - Directly producing native binary (machine code, assembly, raw bytes) and byte-code instead of tokens in source code [42], [43], [44]. Transformer-based generative models can be designed and trained on specific CPU instructions, opcodes, and assembly to create a natural binary interface or apply optimizations with assistance from verification software. However, this approach is not reliable: single-bit hallucinations can cause crashes, debugging is difficult, writing source code which further produced to binary is more practical due to availability of existing training datasets and deterministic compilers. However, these models are applicable for reverse engineering and deobfuscation tasks, specifically during the generation of intermediate representation (IR) in Ghidra.
Open models for transparency and reproducibility	Companies like OpenAI and Anthropic lack transparency regarding their methodologies and datasets. Generally, they utilize several data sources for training: internal data from their own platforms and clients, data provided by vendors and enterprise partners, and web-crawled data. The last category is particularly difficult to index or remove, as many American companies do not adhere to GDPR (The General Data Protection Regulation) or European data protection practices. Also, there are common methods to prevent threats

	of rogue or poisoned models: using verified open weights models for reproducibility or contributions, cryptographic verification (hashing and keys), software bills of materials, and rigorous testing.
Small and optimized models	- Using integer-based models instead of floats and 1-bit models like BitNet [45]. - Applying SLMs (small language models) instead of LLMs (large language models) for specialized tasks: mathematics proofs, seq2seq (sequence to sequence), NER (named entity recognition for structured extraction of entities or actions), classification, formatting, coding snippets and autocompletions, automating repetitive actions with agentic AI, searching data, and so on. For example, it is possible to construct a pipeline that evaluates the complexity of a given task and redirects it to a smaller model, achieving reasonable generation speed and sufficient output quality [46]. - Lossless compression of model weights for "data at rest" and decompression for "data in use". - Quantization for pre-trained models [47]; fine-tuning with Low-Rank Adaptation (LoRA) to reduce redundant parameters [48]; KV caching for distributed inference to reduce time to first token [49], [50]; TurboQuant as vector quantization algorithm [51]; multi-dimensional partitioning and multiquery attention [52]; and so on.

C. *Security of execution environment*

Security in distributed systems is critical for stabilizing nodes and preventing localized issues from propagating across the network. Firstly, the OWASP Application Security Verification Standard (ASVS) provides measurable and actionable security scores for web-based services [53]. NIST highlights security-focused configuration management as a critical component of systematic monitoring [54]. Secondly, mobile operating systems such as iOS and Android offer robust default capabilities through restrictive application sandboxing and a strict permission model [55], [56], [57]. Moreover, the concept of Trusted Execution Environments (TEEs) introduces hardware and software isolated execution and confidential computing for sensitive operations [58], [59]. For example, [60] evaluated WebAssembly (WASM) as a viable execution runtime that provides a secure and platform-agnostic sandbox. However, the context of malware detection and dependency analysis still remains essential for evaluating untrusted applications and third-party libraries [61], [62]. Thirdly, [63] suggest that trusted computing and attestations are required for maintaining protocol adherence at the hardware level and mitigating risks from lower-level kernel exploits. These measures include TPM chips as a hardware root-of-trust, hardware identification such as MAC addresses, and specialized firmware or software to verify node integrity.

III. Methodology

A. *Concept of a distributed network*

We specified mechanisms of consensus and rules by which network participants as nodes reach agreement or are configured for correct operations. Our network is based on PoUW for solving useful computing and other defined tasks. There is a primary abstraction termed a "unit of work" that is achieved by completion of relatively "valuable or useful work". In this regard, the network has two roles of participants which determine the work usefulness and their intrinsic value:

- Providers - can create tasks by publishing them to the network and determine the validation criteria for received results.
- Contractors - can select available tasks from the network and perform them.

During the prototyping and early stages of the system, we consider a limited set of predefined/fixed types of tasks, including data annotation, dataset storage, model inference, and model training. Future iterations of the network will support custom general-purpose and computational tasks that are defined and implemented on the software side via smart contracts:

- ASIC-resistance due to the diversity of tasks that cannot be solved using a single method.
- Preserving the value of the completed work, compliance with specified criteria, and other features.
- Providing additional packages and logic (e.g., visual modifications and functionalities) due to certain specifics of the task.

Fig. 1 demonstrates that our consensus is an intermediate solution that varies dynamically from task to task. Also, it is possible to combine PoW and PoUW in particular proportions to maintain baseline security guarantees and ensure a seamless transition from legacy networks (e.g., a distribution of 20% for PoW and 80% for PoUW).

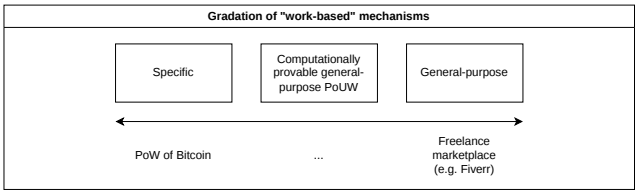

Fig. 1. Gradation of work-based consensus mechanisms: Specific one (on the left) is based on a single fixed algorithm or task type. General-purpose one (on the right) is decoupled from computing power, so it is too abstract and difficult to verify. Our PoUW (on the center) is based on computationally provable tasks, but also allows custom general-purpose tasks on demand.

If an organization needs to train an AI model or store public datasets without maintaining their own IT infrastructure, they can acquire tokens to request on-demand computing power by publishing a smart contract that broadcasts their requirements to the network. Then, someone can accept or join the task in a similar way to quests in multiplayer games, as illustrated in Fig. 2. Contractors can choose tasks manually or enable an automatic mode to match them based on a specific algorithm or preferences.

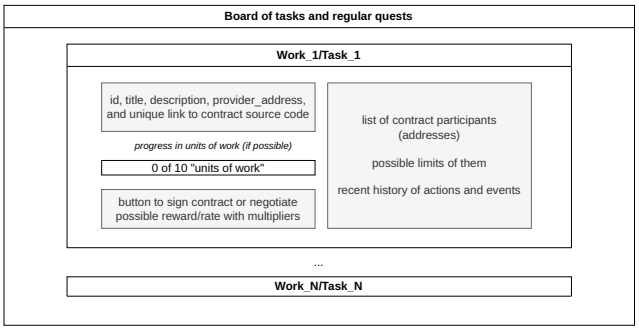

Fig. 2. Market-driven mechanisms for providers and contractors. In this context, tasks are valued relative to a set rate per unit of work in tokens.

Furthermore, we propose a concept of fairness, which is achieved through a limited total supply of tokens and an inflation mechanism designed to prevent holding and market monopolization:

- If tokens are not spent or transferred within a specific timeframe, they gradually burn from the wallet, returning to the limited total supply, as illustrated in Fig. 3.
- A single wallet has a maximum limit of tokens.
- Network participants have the option to voluntarily burn some amount of tokens to refill total supply.

Initial tokens in the network is generated via a Genesis Task, which handles minting and system bootstrapping, as otherwise token acquisition would be impossible.

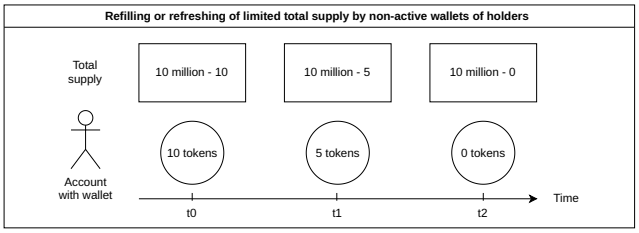

Fig. 3. Refilling of limited total supply by non-active wallets of holders or by burning tokens from their wallets.

From a technical standpoint, we want to build upon the foundations of IPFS and Ethereum, while introducing proprietary extensions and enhancements. As shown in Fig. 4, a node is any single component within the network that executes our certified cross-platform software (daemon, browser-based, or native) and establishes connections by adhering to the consensus protocol with underlying networking stack.

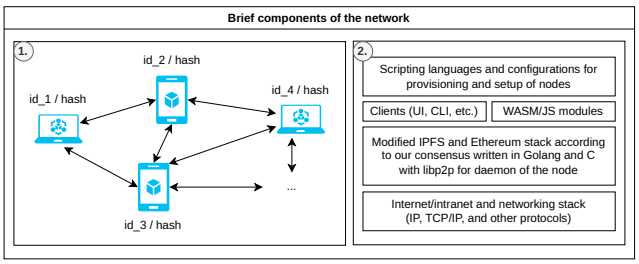

Fig. 4. Components of the network: (1). Visualization for portion of heterogeneous nodes with unique identifiers or hashes in the P2P network. (2). Separation of concerns with smart contracts in WASM.

The first layer of proficency with our software is a rich-client wrapper featuring gamification elements, designed for streamlined auto-updates and efficient release publishing, as shown in Fig. 5. It incorporates active token farming and rewards in the form of in-game assets (skins, models, boosts, and other content). It act as an addictive simulator or "farming-oriented" game, where published tasks and actions are seamlessly integrated into mini-games and quests. The second layer focuses on passive utilizing computing resources (storage, CPU, memory, etc.), operating as a legitimate miner. It features a slider for manual load adjustment or automatic dynamic tuning based on available resources. For instance, the system calculates the necessary overhead while maintaining a base cap of approximately 20% of total device load to prevent hardware wear or excessive battery drain. This layer also involves developing custom smart contracts and is tailored toward technical audience.

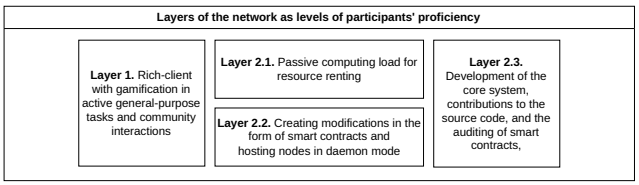

Fig. 5. Layers of the network as levels of participants' proficiency.

As shown in Fig. 6, the system allows connections to various subnets, offering a choice between the main public realm (which requires an internet connection) and alternative subnets. This architecture loosely resembles the mechanics of Usenet groups, which organized multiple nodes based on shared interests or specific characteristics. Furthermore, to ensure fault tolerance against system crashes and data transmission issues, data is backed up to a persistent SQLite database and automatically synchronized upon reconnection.

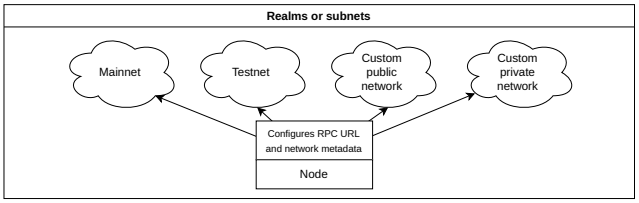

Fig. 6. Nodes can configure connections to specific realms or subnets by specifying the network's RPC URL and associated metadata.

B. *Practical analysis on the dataset of Android-compatible devices*

We gathered a dataset from public sources consisting of Android-compatible smartphones ranging from Android version 1 to 16 and beyond, containing 1745 unique models after pre-processing. The primary objective was to study security issues, unique features, and general complexities of utilizing such devices as low-end nodes. Firstly, detailed description of the dataset is provided in Table 4. Secondly, data collection was carried out manually and through Python scripts. Thirdly, visualizations were rendered in Matplotlib and processed using Pandas, while the raw data was saved in a structured JSON format. Finally, the remaining details are described in more depth in the repository containing our source materials [64].

TABLE IV. DESCRIPTION OF DATASET WITH ANDROID SMARTPHONES

Field	*Data type and description*
model	str; the formal or consumer name of the device model
developer	str; the manufacturer or vendor that developed, sells, or brands the specific device instance
release_date	datetime; the date of public release for consumer purchase
android_version	str; the stock version at release or the default minimal version supported by the smartphone in normalized form ("major.minor.patch")
status	str; one of the possible statuses for the smartphone's Android version (latest, unknown, supported, or unsupported)
latest_security_patch_date	datetime; the date of the latest / last security patch released for the smartphone's Android version

IV. RESULTS AND DISCUSSIONS

A. *Analysis results and statistical charts from the dataset*

Fig. 7 represents the major Android versions across the collected dataset of smartphones, where popularity peaks are visible at versions 4, 10, and 11. Unclear decline is also noticeable starting from version 11, but it is most likely related to the limited sample size in our dataset, possible drop in interest in Android devices, a reduction in smartphone model segments, monopolization around a limited number of manufacturers, or limited support for the latest Android versions from custom distributions (e.g., MIUI, OneUI, etc.). The graph illustrates that there are 980 devices with Android versions ranging from 1 to 9 out of the 1742 shown in total, indicating a massive volume of old or legacy devices that require attention in order to find a practical use for them.

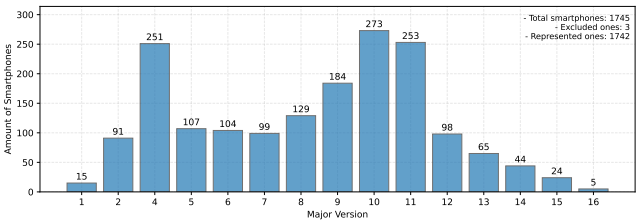

Fig. 7. Major Android versions across smartphone models, excluding certain smartphones due to inconsistencies in their versioning semantics and unconfirmed / unknown Android versions on them.

Fig. 8 groups devices by their versions relative to support for Termux which is a terminal emulator for Android that allows running ported binaries, scripts, or acting similar to Linux distribution environment, rather than creating a separate application with calls to the Android API. Notably, the majority of devices of 67.4% fall into the compatible category we are targeting, while the remaining 32.6% represent a problematic segment that is difficult to work with and maintain.

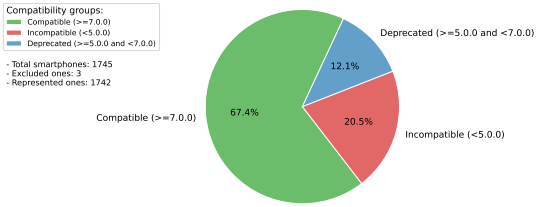

Fig. 8. Compatibility with Termux depending on Android versions (% of smartphones). Deprecated status formally means that it works on older versions without the Termux package ecosystem, which requires building packages from source. Other incompatibles might be mitigated by manually porting Termux with wrappers over the Android API for specific old versions, but without proper support or patches for kernel vulnerabilities.

Fig. 9 and 10 address security and the level of support these devices receive in terms of Android version and kernel updates. The Android version status for 1669 devices is "unsupported", meaning the version is not the latest and has reached end-of-life which is largely applies to all versions except 15 and 16. Of course, this does not rule out the existence of backports from Google when necessary, nor does it exclude support models provided by manufacturers for custom firmware. Fig. 10 expands on this understanding by detailing critical facts regarding smartphone releases, with a peak in 2019 and 2020. Furthermore, the security aspect is examined from the perspective of the latest security patch, which reveals that approximately 66.78%, amounting to 989 out of 1481 devices, received their latest security patches for their Android version three or more years ago.

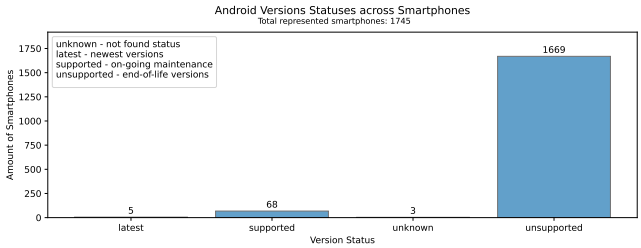

Fig. 9. Device statuses, where some are displayed as “unknown” because the Android versions on them were unidentified.

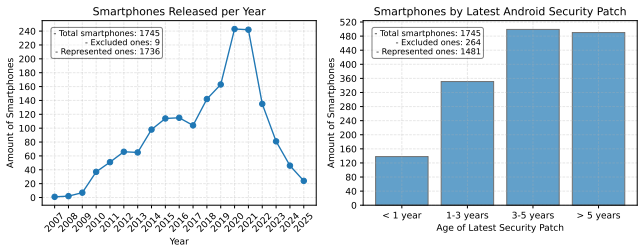

Fig. 10. Amount of smartphone models released per year and their latest Android security patches.

B. *Issues of distributed networks and state of security*

Firstly, the utilization of legacy Android smartphones for a distributed network is a complex set of physical and systemic challenges. For instance, these devices struggle to maintain a charge under load for extended periods due to severe battery wear and overall hardware degradation. Additionally, in some jurisdictions, utilizing such devices is prohibited under European Union Energy Label standards because they are potential fire hazards and are significantly less efficient, resulting in higher power consumption with lower performance output. Moreover, outdated firmware and unsupported Android versions leave these devices vulnerable to unpatched exploits at both the OS and kernel levels, requiring significant technical expertise to mitigate through methods such as rooting or installing custom distributions. Consequently, connecting them to the public web poses a substantial security risk. To ensure safety, it is necessary to restrict their use to air-gapped, isolated local environments and private networks.

Secondly, Fig. 11 demonstrates that distributed computing can be extended to other types of hardware and devices, such as microcontrollers and single-board computers (e.g., ESP-32, Raspberry Pi, Jetson Nano, etc.). These alternatives lack the operating system overhead that consumes some amount of memory on smartphones, but this constraint can be further mitigated by expanding capacity and offloading to a swap partition. Also, self-hosters of nodes have to take into account factors of wired connections and dedicated network cards because the process of data transmission represents a significant bottleneck due to the high volume of data and the inherent speed limitations of standard LAN (local area network) in home routers when utilizing RPC for inter-device communication.

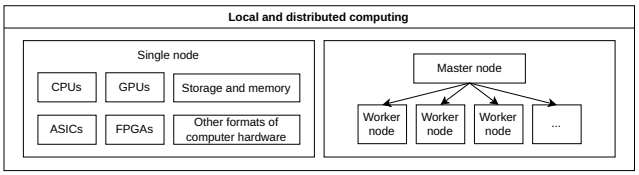

Fig. 11. Local and distributed computing on heterogeneous hardware.

Thirdly, the practical application of smartphones extends beyond our primary system, leveraging Termux and proot-distro for .deb packages and ADB (Android Debug Bridge) connections to enable several use cases:

- Custom inference servers - utilizing the llama-cpp engine to host local AI models distributed via RPC.
- Decentralized storage - hosting open-source GitHub repositories as persistent data within the IPFS network.
- Web hosting - serving web resources on individual devices using Nginx as a load balancer and IPFS for content delivery.
- General distributed computing - executing general-purpose computations (e.g., using Ray framework).
- Cryptocurrency mining - running specialized software like cpuminer with a stratum server (e.g., for Monero or Bitcoin).

Fourthly, the decentralization of our network attracts bad actors who may utilize it for malicious purposes, such as C2 (Command and Control) infrastructure or illegal operations, because the distributed nature makes it nearly impossible to moderate or enforce traditional compliance. Also, it leads to critical issues regarding data spamming and node synchronization that persist even with the implementation of limits and additional rules. There are significant hurdles regarding data loss and reliability that can only be fully identified and resolved through practical implementation and debugging. The security of WASM-based smart contracts

remains a major challenge because auditing them is difficult, making it unclear how to guarantee computational correctness or moderate against malicious actors who will inevitably seek to bypass protective measures and abuse network resources [65].

Fifthly, the system faces significant risks regarding tokenomics and asset abuse. Potential exploit schemes similar to cryptocurrency tumblers could be used to cycle tokens between accounts, while bot farms might deploy empty contracts to manipulate network activity. These issues might be mitigated by imposing limits on transfers or operations. However, social factors remain unavoidable as actors may perform actions outside the system's predefined framework in the real world. To address this, the network may require external validators to assess task quality and moderate the network, or cooperation with specific local state and government regulators.

Lastly, the proof-of-burn mechanic might discourage certain participants due to the loss of accumulated wealth, necessitating future consideration of alternatives justification for the economic feasibility of this model. Our original intent was to incentivize the constant circulation of capital rather than passive holding, thereby reducing the influence of actors with large holdings over the network's control. Additionally, we considered an investment option involving registered public funds such as universities or charities. In this model, users could transfer tokens to these entities to reduce or prevent token burning for a specific period, reflecting a concept similar to universal basic income. This approach may be revised based on the overall viability of the system, especially in scenarios where the total supply is depleted while existing tokens have not yet been burned, leaving no available liquidity for new participants.

V. Conclusion

The necessity of repurposing legacy and low-end devices during times of hardware crisis, combined with the goal of replacing centralized control with decentralized alternatives, remains a critical objective. Our results indicate that further measures must be taken to ensure system stability and safety, because certain categories of smartphones inevitably become electronic waste due to their low computational utility and a lack of critical software updates and security patches. Moreover, upcoming technical challenges include stricter Android security policies that may require device verification or disallow non-certified applications from running, potentially restricting execution to Google Play-certified software only. Additionally, the difficulty of running systemd on Android, even with termux-services, further complicates porting standard Linux programs to these devices.

Future work will focus on the practical implementation of custom-engineered smartphone and secondary device farms of modular 3D-printed server racks as docking stations equipped with power wiring and active cooling to prevent overheating and rapid discharge. Also, there is a clear opportunity to extend inference capabilities by implementing meta-prompting and self-configuring distributed inference loops using llama-cpp and llama-bench, specifically optimized for low-resource hardware. Furthermore, we aim to collect more detailed data across diverse low-end device categories, focusing on application compatibility, performance benchmarks, and specific hardware characteristics.

References

[1] V. Hassija, S. Zeadally, I. Jain, A. Tahiliani, V. Chamola, and S. Gupta, "Framework for determining the suitability of blockchain: Criteria and issues to consider," *Trans. Emerg. Telecommun. Technol.*, vol. 32, no. 10, p. e4334, Oct. 2021, doi: 10.1002/ett.4334.

[2] Y. Sompolinsky, S. Wyborski, and A. Zohar, "PHANTOM GHOSTDAG: a scalable generalization of Nakamoto consensus: September 2, 2021," in *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies*, Arlington Virginia: ACM, Sep. 2021, pp. 57–70. doi: 10.1145/3479722.3480990.

[3] B. Chen *et al.*, "A Comprehensive Survey of Blockchain Scalability: Shaping Inner-Chain and Inter-Chain Perspectives," Sep. 04, 2024, *arXiv*: arXiv:2409.02968. doi: 10.48550/arXiv.2409.02968.

[4] L. Zhang, T. Wang, and S. C. Liew, "Speeding up Block Propagation in Blockchain Network: Uncoded and Coded Designs," Jan. 02, 2021, *arXiv*: arXiv:2101.00378. doi: 10.48550/arXiv.2101.00378.

[5] R. Lavin, X. Liu, H. Mohanty, L. Norman, G. Zaarour, and B. Krishnamachari, "A Survey on the Applications of Zero-Knowledge Proofs," Aug. 01, 2024, *arXiv*: arXiv:2408.00243. doi: 10.48550/arXiv.2408.00243.

[6] X. Tang, L. Shi, X. Wang, K. Charbonnet, S. Tang, and S. Sun, "Zero-Knowledge Proof Vulnerability Analysis and Security Auditing".

[7] E. Jones and W. Knottenbelt, "Towards Automating Blockchain Consensus Verification with IsabeLLM," Jan. 12, 2026, *arXiv*: arXiv:2601.07654. doi: 10.48550/arXiv.2601.07654.

[8] Y. I. Alzoubi and A. Mishra, "Green blockchain – A move towards sustainability," *J. Clean. Prod.*, vol. 430, p. 139541, Dec. 2023, doi: 10.1016/j.jclepro.2023.139541.

[9] Y. Tomatsu and W. Han, "Bitcoin and Renewable Energy Mining: A Survey," *Blockchains*, vol. 1, no. 2, pp. 90–110, Dec. 2023, doi: 10.3390/blockchains1020007.

[10] F. Hoffmann, "Challenges of Proof-of-Useful-Work (PoUW)," Sep. 08, 2022, *arXiv*: arXiv:2209.03865. doi: 10.48550/arXiv.2209.03865.

[11] A. Lihu, J. Du, I. Barjaktarevic, P. Gerzanics, and M. Harvilla, "A Proof of Useful Work for Artificial Intelligence on the Blockchain," Jan. 25, 2020, *arXiv*: arXiv:2001.09244. doi: 10.48550/arXiv.2001.09244.

[12] R. Pass and E. Shi, "FruitChains: A Fair Blockchain," in *Proceedings of the ACM Symposium on Principles of Distributed Computing*, Washington DC USA: ACM, Jul. 2017, pp. 315–324. doi: 10.1145/3087801.3087809.

[13] S. D. Lerner, "APoW: Auditable Proof-of-Work Against Block Withholding Attacks," Jan. 08, 2026, *arXiv*: arXiv:2601.02496. doi: 10.48550/arXiv.2601.02496.

[14] A. Stoz and T. Debatty, "A decentralized architecture for package manager based on the blockchain".

[15] M. N. Ince, M. Ak, and M. Gunay, "Blockchain Based Distributed Package Management Architecture," in *2020 5th International Conference on Computer Science and Engineering (UBMK)*, Diyarbakir, Turkey: IEEE, Sep. 2020, pp. 238–242. doi: 10.1109/UBMK50275.2020.9219374.

[16] F. Z. D. N. Costa, R. J. G. B. De Queiroz, G. P. Bittencourt, and L. Teixeira, "Distributed Repository for Software Packages Using Blockchain," *IEEE Access*, vol. 10, pp. 112502–112514, 2022, doi: 10.1109/ACCESS.2022.3216569.

[17] E. J. Scheid, M. Keller, M. F. Franco, and B. Stiller, "BUNKER: A Blockchain-based trUsted VNF pacKagE Repository," in *Economics of Grids, Clouds, Systems, and Services*, vol. 11819, K. Djemame, J. Altmann, J. Á. Bañares, O. Agmon Ben-Yehuda, and M. Naldi, Eds., in Lecture Notes in Computer Science, vol. 11819. , Cham: Springer International Publishing, 2019, pp. 188–196. doi: 10.1007/978-3-030-36027-6_16.

[18] M. Hammad *et al.*, "Blockchain-Based Decentralized Architecture for Software Version Control," *Appl. Sci.*, vol. 13, no. 5, p. 3066, Feb. 2023, doi: 10.3390/app13053066.

[19] N. Nizamuddin, K. Salah, M. Ajmal Azad, J. Arshad, and M. H. Rehman, "Decentralized document version control using ethereum blockchain and IPFS," *Comput. Electr. Eng.*, vol. 76, pp. 183–197, Jun. 2019, doi: 10.1016/j.compeleceng.2019.03.014.

[20] G. Giamouridis, B. Kang, and L. Aniello, "Blockchain-based DNS: Current Solutions and Challenges to Adoption".

[21] J. Sevilla, L. Heim, A. Ho, T. Besiroglu, M. Hobbhahn, and P. Villalobos, "Compute Trends Across Three Eras of Machine Learning," in *2022 International Joint Conference on Neural Networks (IJCNN)*, Jul. 2022, pp. 1–8. doi: 10.1109/IJCNN55064.2022.9891914.

[22] M. J. Absar *et al.*, "Hexagon-MLIR: An AI Compilation Stack For Qualcomm's Neural Processing Units (NPUs)," Feb. 23, 2026, *arXiv*: arXiv:2602.19762. doi: 10.48550/arXiv.2602.19762.

[23] A. Jelinčič *et al.*, “An efficient probabilistic hardware architecture for diffusion-like models,” Dec. 10, 2025, *arXiv*: arXiv:2510.23972. doi: 10.48550/arXiv.2510.23972.

[24] “The path to ubiquitous AI,” Taalas. Accessed: Apr. 11, 2026. [Online]. Available: https://taalas.com/the-path-to-ubiquitous-ai/

[25] H. Zhao *et al.*, “Exponential quantum advantage in processing massive classical data,” Apr. 08, 2026, *arXiv*: arXiv:2604.07639. doi: 10.48550/arXiv.2604.07639.

[26] P. L. McMahon, “The physics of optical computing,” *Nat. Rev. Phys.*, vol. 5, no. 12, pp. 717–734, Oct. 2023, doi: 10.1038/s42254-023-00645-5.

[27] P. Li, M. J. Islam, and S. Ren, “A Case Study of Environmental Footprints for Generative AI Inference: Cloud versus Edge”.

[28] L. Amatuni, C. Clemm, B. Sprecher, A. Tukker, and J. M. Mogollón, “Second-hand smartphones reduce carbon emissions, yet shorter use times limit actual gains,” *Commun. Earth Environ.*, vol. 7, no. 1, p. 148, Jan. 2026, doi: 10.1038/s43247-025-03170-8.

[29] M. Moran, J. Balladini, D. Rexachs, and E. Luque, “Checkpoint and Restart: An Energy Consumption Characterization in Clusters,” Sep. 03, 2024, *arXiv*: arXiv:2409.02214. doi: 10.48550/arXiv.2409.02214.

[30] J. Lee *et al.*, “On-Device Neural Net Inference with Mobile GPUs,” Jul. 03, 2019, *arXiv*: arXiv:1907.01989. doi: 10.48550/arXiv.1907.01989.

[31] M. Oellermann *et al.*, “Open Hardware in Science: The Benefits of Open Electronics,” *Integr. Comp. Biol.*, vol. 62, no. 4, pp. 1061–1075, Oct. 2022, doi: 10.1093/icb/icac043.

[32] DeepSeek-AI *et al.*, “DeepSeek-V3 Technical Report,” Feb. 18, 2025, *arXiv*: arXiv:2412.19437. doi: 10.48550/arXiv.2412.19437.

[33] “Single Event Burnout Hardened High-power Diamond Devices.” Accessed: Apr. 11, 2026. [Online]. Available: https://techport.nasa.gov/projects/154516

[34] Q. Chen, J. Wang, and J. Lin, “Generative AI exacerbates the climate crisis,” *Science*, vol. 387, no. 6734, pp. 587–587, Feb. 2025, doi: 10.1126/science.adt5536.

[35] A. Kashyap, “The Adoption of Green Programming Languages as a Promising Approach to Improve Computing’s Sustainability,” *Int. J. High Sch. Res.*, vol. 6, no. 5, pp. 18–24, May 2024, doi: 10.36838/v6i5.4.

[36] R. Pereira *et al.*, “Energy efficiency across programming languages: how do energy, time, and memory relate?,” in *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering*, Vancouver BC Canada: ACM, Oct. 2017, pp. 256–267. doi: 10.1145/3136014.3136031.

[37] N. van Kempen, H.-J. Kwon, D. T. Nguyen, and E. D. Berger, “It’s Not Easy Being Green: On the Energy Efficiency of Programming Languages,” Oct. 03, 2025, *arXiv*: arXiv:2410.05460. doi: 10.48550/arXiv.2410.05460.

[38] DeepSeek-AI *et al.*, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” 2025, *arXiv*. doi: 10.48550/ARXIV.2501.12948.

[39] M. A. Hakim, “Brevity Constraints Reverse Performance Hierarchies in Language Models,” Mar. 11, 2026, *arXiv*: arXiv:2604.00025. doi: 10.48550/arXiv.2604.00025.

[40] T. Ridnik, D. Kredo, and I. Friedman, “Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering,” Jan. 16, 2024, *arXiv*: arXiv:2401.08500. doi: 10.48550/arXiv.2401.08500.

[41] V. Paramanayakam, A. Karatzas, I. Anagnostopoulos, and D. Stamoulis, “Less is More: Optimizing Function Calling for LLM Execution on Edge Devices,” Nov. 23, 2024, *arXiv*: arXiv:2411.15399. doi: 10.48550/arXiv.2411.15399.

[42] J. Kallini, S. Murty, C. D. Manning, C. Potts, and R. Csordás, “MrT5: Dynamic Token Merging for Efficient Byte-level Language Models,” Apr. 02, 2025, *arXiv*: arXiv:2410.20771. doi: 10.48550/arXiv.2410.20771.

[43] S. Wu, X. Tan, Z. Wang, R. Wang, X. Li, and M. Sun, “Beyond Language Models: Byte Models are Digital World Simulators,” Feb. 29, 2024, *arXiv*: arXiv:2402.19155. doi: 10.48550/arXiv.2402.19155.

[44] A. Pagnoni *et al.*, “Byte Latent Transformer: Patches Scale Better Than Tokens,” Dec. 13, 2024, *arXiv*: arXiv:2412.09871. doi: 10.48550/arXiv.2412.09871.

[45] S. Ma *et al.*, “The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits,” Feb. 27, 2024, *arXiv*: arXiv:2402.17764. doi: 10.48550/arXiv.2402.17764.

[46] P. Belcak *et al.*, “Small Language Models are the Future of Agentic AI,” Sep. 15, 2025, *arXiv*: arXiv:2506.02153. doi: 10.48550/arXiv.2506.02153.

[47] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” Mar. 22, 2023, *arXiv*: arXiv:2210.17323. doi: 10.48550/arXiv.2210.17323.

[48] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models,” Oct. 16, 2021, *arXiv*: arXiv:2106.09685. doi: 10.48550/arXiv.2106.09685.

[49] H. Matsutani, N. Matsuda, and N. Sugiura, “Accelerating Local LLMs on Resource-Constrained Edge Devices via Distributed Prompt Caching,” Feb. 26, 2026, *arXiv*: arXiv:2602.22812. doi: 10.48550/arXiv.2602.22812.

[50] K. Staniszewski and A. Łańcucki, “KV Cache Transform Coding for Compact Storage in LLM Inference,” Mar. 11, 2026, *arXiv*: arXiv:2511.01815. doi: 10.48550/arXiv.2511.01815.

[51] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, “TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate,” Apr. 28, 2025, *arXiv*: arXiv:2504.19874. doi: 10.48550/arXiv.2504.19874.

[52] R. Pope *et al.*, “Efficiently Scaling Transformer Inference,” Nov. 09, 2022, *arXiv*: arXiv:2211.05102. doi: 10.48550/arXiv.2211.05102.

[53] S.-F. Wen and B. Katt, “A quantitative security evaluation and analysis model for web applications based on OWASP application security verification standard,” *Comput. Secur.*, vol. 135, p. 103532, Dec. 2023, doi: 10.1016/j.cose.2023.103532.

[54] A. Johnson, K. Dempsey, R. Ross, S. Gupta, and D. Bailey, “Guide for security-focused configuration management of information systems,” National Institute of Standards and Technology, Gaithersburg, MD, NIST SP 800-128, Oct. 2019. doi: 10.6028/NIST.SP.800-128.

[55] S. Khan, A. Yusuf, M. Haider, K. Thirunavukkarasu, P. Nand, and M. K. Imam Rahmani, “A Review of Android and iOS Operating System Security,” in *2022 ASU International Conference in Emerging Technologies for Sustainability and Intelligent Systems (ICETSIS)*, Manama, Bahrain: IEEE, Jun. 2022, pp. 67–72. doi: 10.1109/ICETSIS55481.2022.9888847.

[56] G. S. Tuncay, “Android Permissions: Evolution, Attacks, and Best Practices,” *IEEE Secur. Priv.*, vol. 22, no. 6, pp. 40–49, Nov. 2024, doi: 10.1109/MSEC.2024.3461629.

[57] S. Laurén, S. Rauti, and V. Leppänen, “A Survey on Application Sandboxing Techniques”.

[58] P. Jauernig, A.-R. Sadeghi, and E. Stapf, “Trusted Execution Environments: Properties, Applications, and Challenges,” *IEEE Secur. Priv.*, vol. 18, no. 2, pp. 56–60, Mar. 2020, doi: 10.1109/MSEC.2019.2947124.

[59] M. Ye *et al.*, “Position Paper: From Confidential Computing to Zero Trust, Come Along for the (Bumpy?) Ride,” in *Proceedings of the 13th International Workshop on Hardware and Architectural Support for Security and Privacy*, Austin TX USA: ACM, Nov. 2024, pp. 19–27. doi: 10.1145/3696843.3696848.

[60] M. Has, T. Xiong, F. B. Abdesslem, and M. Kušek, “WebAssembly on Resource-Constrained IoT Devices: Performance, Efficiency, and Portability,” 2025, *arXiv*. doi: 10.48550/ARXIV.2512.00035.

[61] B. Amro, “Malware Detection Techniques for Mobile Devices,” *Int. J. Mob. Netw. Commun. Telemat.*, vol. 7, no. 4/5/6, pp. 01–10, Dec. 2017, doi: 10.5121/ijmnct.2017.7601.

[62] J. Samhi, T. F. Bissyandé, and J. Klein, “AndroLibZoo: A Reliable Dataset of Libraries Based on Software Dependency Analysis,” in *Proceedings of the 21st International Conference on Mining Software Repositories*, Apr. 2024, pp. 32–36. doi: 10.1145/3643991.3644866.

[63] M. Bartock *et al.*, “Hardware-enabled security : enabling a layered approach to platform security for cloud and edge computing use cases,” National Institute of Standards and Technology (U.S.), Gaithersburg, MD, NIST IR 8320, May 2022. doi: 10.6028/NIST.IR.8320.

[64] N. Tendikov, NoyanTM/iccsfai-2026_research-paper. (Apr. 05, 2026). Accessed: Apr. 20, 2026. [Online]. Available: https://github.com/NoyanTM/iccsfai-2026_research-paper

[65] W. Chen, Z. Sun, H. Wang, X. Luo, H. Cai, and L. Wu, “WASAI: uncovering vulnerabilities in Wasm smart contracts,” in Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual South Korea: ACM, Jul. 2022, pp. 703–715. doi: 10.1145/3533767.3534218.